Mini AI BI Assistant

Runbook: what each part is, how to run it, and the logic behind every answer

What the app does

Business users type a question in plain English. The app works out which database query answers it, runs it, and shows the answer with the figures and SQL behind it.

1. **Ask** "Which region sold the most monitors in May?"
2. **Plan** Rules or an LLM turn the question into a safe query plan
3. **Query** SQLite computes the figures, read-only
4. **Answer** "East, 9 units" plus supporting rows, chart and SQL

51 rows in the CSV	**49** rows loaded
2 rows rejected, with reasons	**84** automated tests

Project map: what is what

Path	What it is
`app/main.py`	FastAPI web app: routes, request validation, error responses, .env loading
`app/db.py`	Loads and cleans the CSV into SQLite; read-only query guard (trusted and untrusted modes)
`app/analytics.py`	Turns a plan into SQL: named intents, QuerySpec builder, text-to-SQL runner, answer text
`app/interpreter.py`	Decides what to run: rule-based parser plus LLM planner, with fallback
`app/llm.py`	Generic LLM client for any OpenAI-compatible API, configured by env vars
`app/static/index.html`	Single-page UI: question box, answer, chart, supporting table, SQL
`data/sales.csv`	Supplied dataset; data/sales.db is rebuilt from it at every start
`tests/`	84 pytest tests: API, components, text-to-SQL, LLM providers
`.env / .env.example`	Your local LLM settings (gitignored) / the committed template
`README.md, AI_USAGE.md`	Setup and architecture docs / how AI was used during development

Running it

Setup, LLM configuration and health checks

Run it in five steps

1. **Open the project folder**

```
cd ai-bi-assistant
```

2. **Create and activate a virtual environment**

```
python3 -m venv .venv
source .venv/bin/activate
```

3. **Install dependencies**

```
pip install -r requirements.txt
```

4. **Optional: configure an LLM**

```
cp .env.example .env    # then edit it
```

5. **Start the server, open the UI**

```
uvicorn app.main:app --reload
# open http://127.0.0.1:8000
```

Run the tests: `pytest -q` (84 tests, about 2 seconds, fully offline)

Choose an LLM with environment variables

Variable	Purpose
LLM_PROVIDER	Preset that fills in the URL: gemini, openai, anthropic, groq, openrouter, ollama
LLM_MODEL	Model name, required (e.g. gemini-3.5-flash)
LLM_API_KEY	API key; falls back to GEMINI_API_KEY, OPENAI_API_KEY, ...
LLM_BASE_URL	Any other OpenAI-compatible endpoint; overrides the preset
LLM_TIMEOUT	Seconds to wait before falling back to the rules (default 30)
LLM_JSON_MODE	none (default), json_object or json_schema
BI_USE_LLM=0	Force rules-only mode even when an LLM is configured

Example .env (Gemini)

```
LLM_PROVIDER=gemini
LLM_MODEL=gemini-3.5-flash
LLM_API_KEY=<your key>
```

- **No LLM set:** runs fully offline on the rules
- **Bad config:** logged; app keeps running on the rules
- **.env is gitignored;** never put a key in .env.example

Check that it is working

`/api/health`

Is it up, and which interpreter?

```
{"status": "ok",
 "interpreter": "gemini/gemini-3.5-flash+rules"}
```

`/api/data-quality`

What loaded and what was rejected?

```
"loaded_rows": 49, "rejected_rows": 2
TXN0034: non-positive Quantity (0)
TXN0051: missing Unit_price
```

`POST /api/ask`

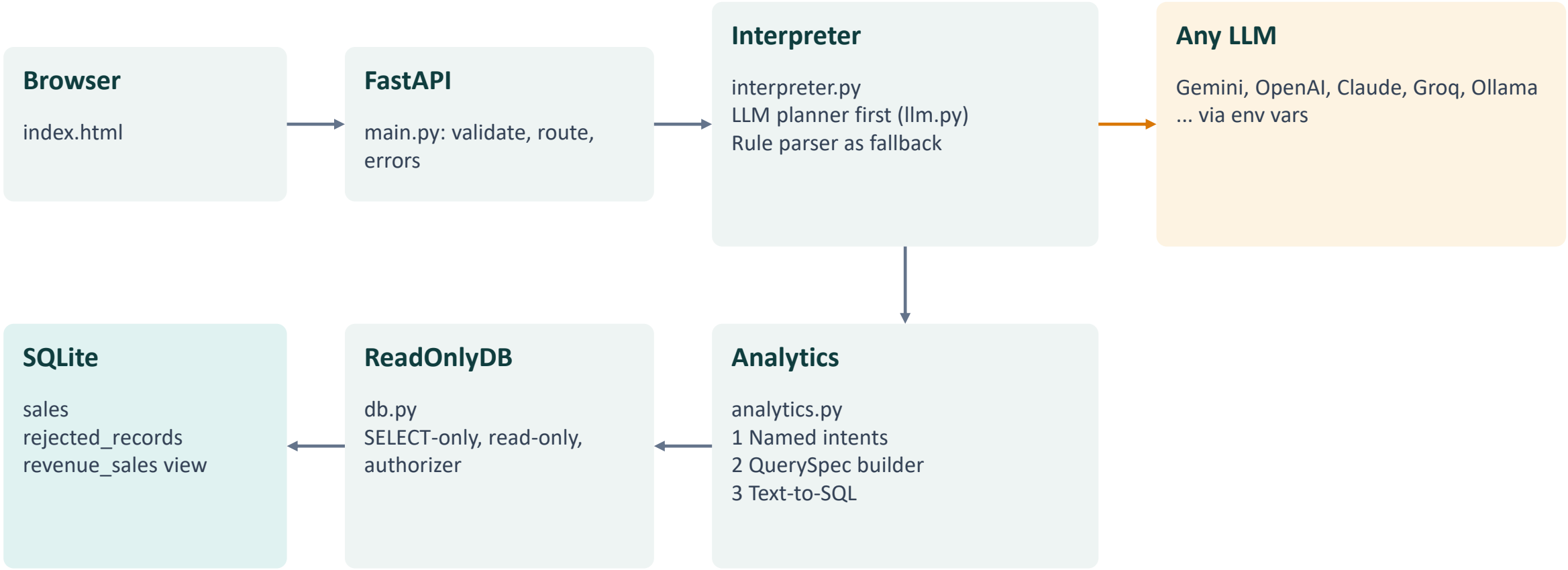
Ask a question from the terminal

```
curl -X POST localhost:8000/api/ask \
  -H 'content-type: application/json' \
  -d '{"question": "total revenue?"}'
```

How it works

Architecture, data layer and the logic that turns a question into SQL

Architecture



Answer path: rows from SQLite → answer text, chart and table → back to the browser as JSON

Data layer: from CSV to SQLite

Each row is checked when the server starts

- Transaction id present and not duplicated
- Product, category, region, channel, status present
- Date in DD-MM-YYYY format, stored as ISO YYYY-MM-DD
- Quantity and unit price are numbers above zero
- Discount is zero or more, and not above the order value
- Status is a known value; text casing is normalised

Revenue = quantity × unit price − discount
Completed orders only. Defined once, in the revenue_sales view.

`sales`

49 valid rows, all statuses, with CHECK constraints

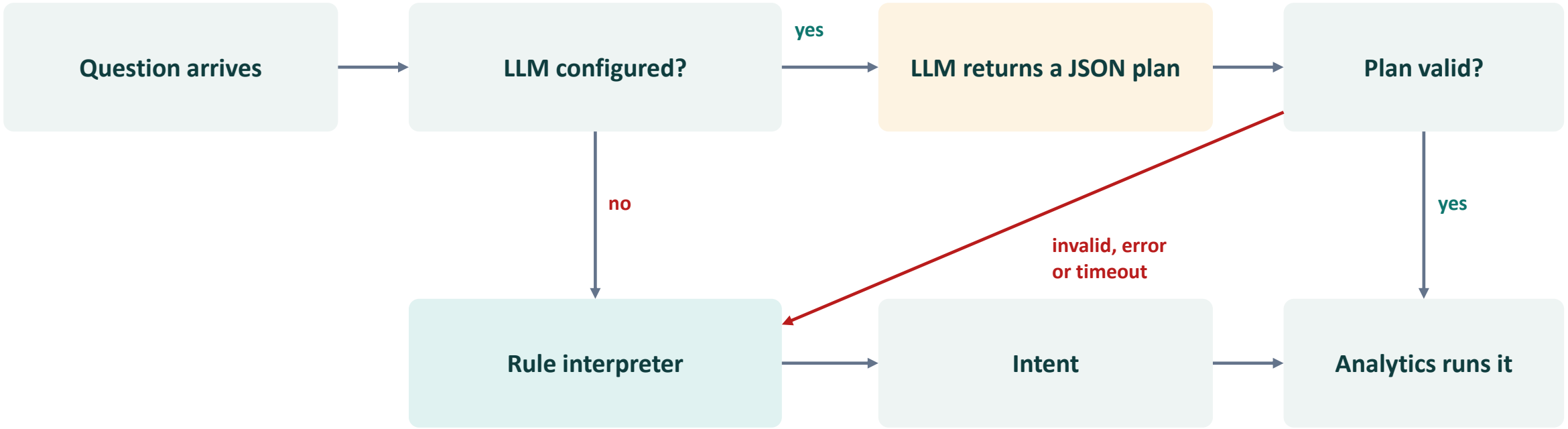
`rejected_records`

2 rejected rows: line number, id, reason, raw values

`revenue_sales (view)`

48 completed orders, plus gross and net revenue columns

Interpreter: who decides what to run



- **Validation:** JSON is checked against a Pydantic schema; an invalid reply gets one retry with the error shown to the model
- **"unsupported"** from either interpreter returns a 422 with a reason and example questions, never a guess

Three levels of query, safest first

1 Named intents

Hand-written SQL with purpose-built answers

- Total revenue
- Top product / top region
- Monthly trend
- Compare two months
- Breakdown by a field

2 QuerySpec builder

SQL assembled only from whitelisted pieces

- Metric: revenue, units, orders, AOV, discount, returns
- Filters: product, region, category, channel, month
- Grouping, highest / lowest, top-N

3 Text-to-SQL

LLM writes SQL; needs an LLM configured

- Only for questions 1 and 2 cannot express
- Runs in strict untrusted mode
- e.g. "How many different products did each region sell?"

Inside the rule interpreter

Step	What it looks for	Example
1. Write guard	delete, drop, update, insert, alter ...	"delete all sales" → refused
2. Out of scope	forecasts, profit, customers, distinct counts, ratios	"predict next month" → refused
3. Unknown names	a word after for/of/from/in that is not in the data	"revenue from Tablet" → refused, known values listed
4. Months	January ... June, 2026-03, "May" but not "may I"	"March vs April" → 2026-03, 2026-04
5. Entities	real values from the database	"monitors in the North" → Monitor, North
6. Metric	units, orders, average order value, discount, returns	"how many units" → units
7. Grouping and ranking	which / by / each, highest, lowest, top-N	"top 3 products" → group product, desc, 3
8. Route	named intent if it fits, otherwise a QuerySpec	plain revenue questions keep their tailored answers

Request lifecycle: POST /api/ask



Validate

Non-empty, ≤ 500 chars, JSON body; else 400

Interpret

LLM or rules produce an intent; unsupported → 422

Check

Months and values exist in the data; else 422 with known values

Query

SQL runs read-only; blocked LLM SQL retries via rules

Phrase

Templated answer; LLM summary only if numbers are grounded

Respond

answer, data rows, chart, SQL, params, assumptions

Example response

```
{"intent": "month_comparison", "interpreted_by": "rules",
 "answer": "Revenue increased from ₹223,900.00 in March 2026 to ₹240,700.00 in April 2026 (+7.5%).",
 "data": {...}, "chart": {...}, "sql": "SELECT ...", "assumptions": [...]}
```

Quality and safety

How wrong or harmful answers are prevented, and how it is tested

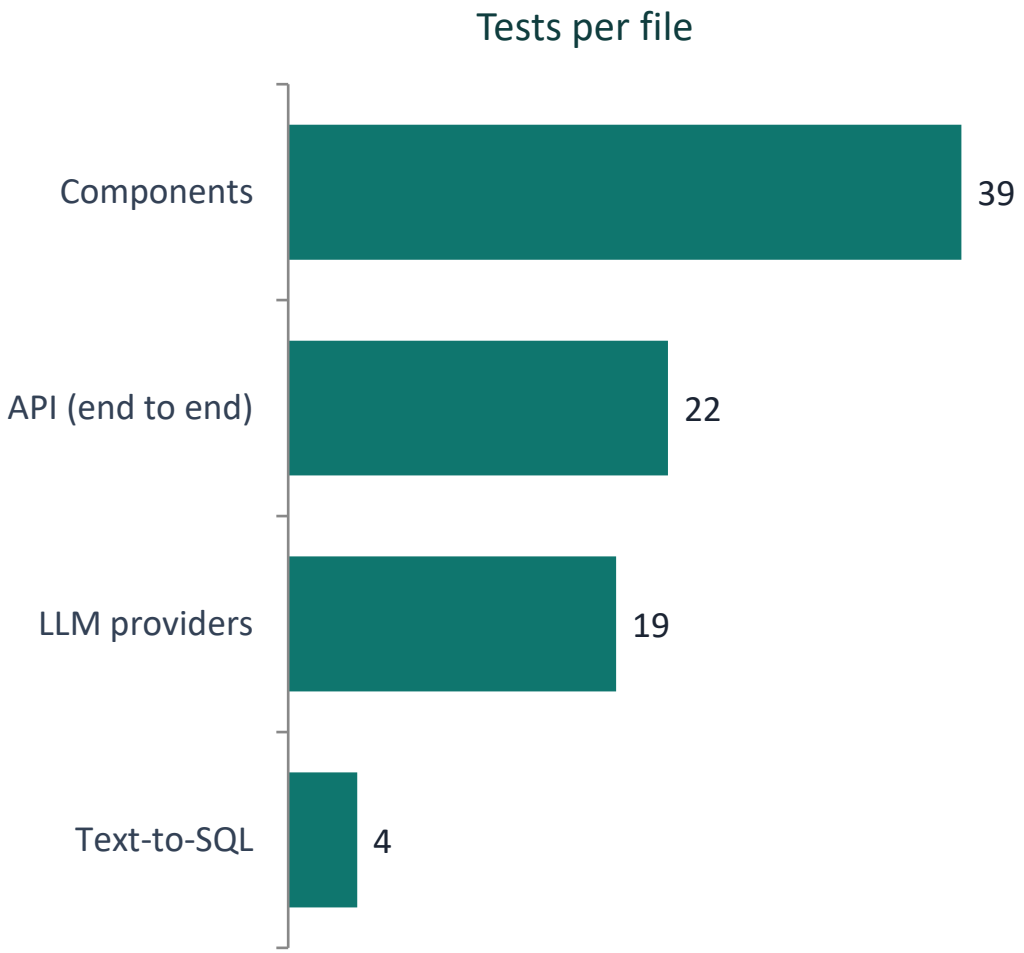
Five layers stop destructive queries

1	**Question guard**	Write verbs are refused before any query is chosen	`interpreter.py`
2	**Plans, not SQL**	Named intents and QuerySpec use whitelisted SQL with bound parameters	`analytics.py`
3	**Static SQL check**	One statement only, must start with SELECT or WITH	`db.validate_sql`
4	**Read-only connection**	SQLite opened with mode=ro refuses any write	`db.ReadOnlyDB`
5	**Authorizer**	Allows only reads; LLM SQL also limited to 2 tables, listed functions, 200 rows, step watchdog	`db._strict_authorizer`

Refuse rather than guess

Question	What the app does	Why
revenue from Tablet	422: "I don't recognise 'tablet'" + known products	Otherwise it would silently return the overall total
Predict next month's revenue	422 unsupported	No forecasting model; the data cannot answer it
Compare January and July	422: no data for July 2026, lists available months	Month outside the data range
How did revenue change between two months?	422: asks which two months	Ambiguous; a clarification, not a guess
delete all sales	422 unsupported; data untouched	Read-only assistant
LLM summary says "up 12%"	Summary discarded; template answer used	12 does not appear in the query result

Testing: 84 automated tests



What is covered

- All five questions from the brief, with exact values
- Invalid input: empty, too long, missing field, not JSON
- Unsupported and destructive questions; data unchanged afterwards
- Edge cases: one-month comparison, month outside the data
- Loader rejection rules and the SQL guard
- LLM: env config, request shape, retry, fallback (fake server)

Run: `pytest -q`

Operations

API reference, troubleshooting, limitations and how it was built

API reference

Endpoint	Returns
`GET /`	The web UI
`POST /api/ask`	Answer, data rows, chart hint, SQL, params, assumptions
`GET /api/health`	Status and active interpreter
`GET /api/data-quality`	Loaded and rejected rows, with reasons
`GET /api/examples`	Example questions for the UI

Status	Code	When
400	invalid_request	Empty, too long or bad body
422	unsupported_question	Cannot map to the data; reason + suggestions
422	cannot_answer	Understood, but e.g. month has no data
500	query_blocked	Safety guard tripped
500	internal_error	Unexpected; logged, not leaked

Troubleshooting

Symptom	Cause	Fix
LLM fails with HTTP 404	Model retired for new users (e.g. gemini-2.5-flash)	Set LLM_MODEL to a current model, e.g. gemini-3.5-flash
HTTP 503 "high demand"	Provider overloaded	Wait, or pick a lighter model; the app falls back to the rules meanwhile
Answers slow, then "rules"	LLM took longer than LLM_TIMEOUT	Raise LLM_TIMEOUT to 60
Health says "rules" despite .env	Missing model or key, or unknown provider	Read the server log line "LLM planner disabled: ..."
ModuleNotFoundError	Virtual environment not active	source .venv/bin/activate, then pip install -r requirements.txt
Address already in use	Port 8000 taken	uvicorn app.main:app --port 8001
Key shows up in git status	Key written to .env.example	Move it to .env (gitignored); rotate the key if it was shared

Known limitations and next steps

Limitations today

- Offline rules: one metric and one grouping per question
- No relative dates ("last month", "Q1")
- Rules miss unusual phrasing; the LLM covers it
- Grounding check catches invented numbers, not wrong claims about right ones
- Currency assumed to be ₹; returns not linked to original sales
- No auth, rate limiting or caching

With another day

- Benchmark the LLM planner on 50+ paraphrased questions
- Multiple metrics and two-level grouping offline
- Relative dates and quarters
- Follow-up questions with conversation memory
- Dockerfile and CI running pytest on every push

How it was built with AI

Built with an AI coding assistant: requirement analysis, design, code, tests. Every output was checked by running it. Bugs found that way:

Bug found	Fix
"delete all sales" returned total revenue	Write-verb guard runs before every other rule
"revenue from Tablet" returned the overall total	Unknown names after for/of/from/in are refused
"How many returned orders?" answered 48	Added returned_orders and returned_value metrics
"different products per region" answered revenue	Distinct counts refused offline; the LLM writes SQL instead
LLM summary could invent numbers	Number-grounding check against the query result
Gemini model retired (404), then overloaded (503)	Model set by env var; automatic fallback to the rules

Git Repository:

https://github.com/varshhhaaa12/Sales_BI_Assistant

Quick reference

```
source .venv/bin/activate                # every new terminal
uvicorn app.main:app --reload            # start; open http://127.0.0.1:8000
pytest -q                                # 84 tests
curl localhost:8000/api/health           # which interpreter is active?
curl localhost:8000/api/data-quality     # what loaded / what was rejected
LLM settings: .env  (LLM_PROVIDER, LLM_MODEL, LLM_API_KEY)
```